Python Interview Preparation — Detailed Guide

For fresher AI/ML, GenAI, Python Developer and Backend interviews

> Purpose: learn the concept, understand the example, then practice explaining it aloud. This document is also structured to become a useful first source for the project's RAG knowledge base.

How to study

For each topic: (1) understand the definition, (2) run or mentally trace the example, (3) answer the interview questions without looking at the notes, and (4) connect the topic to your own projects.

1. Python Fundamentals

What is Python?

Python is a high-level, general-purpose programming language with readable syntax and a large ecosystem.

It is dynamically typed: you normally do not declare a variable's type before assigning an object.

Python is used for web applications, automation, scripting, data analysis, machine learning and AI.

```
name = "Omkar"
age = 21
print(name, age)
```

Is Python interpreted or compiled?

Python source code is normally compiled to bytecode and then executed by the Python virtual machine.

So the interview statement 'Python is only an interpreted language' is an oversimplification.

At interview level, explain the practical execution path rather than arguing only from the label.

Why Python is widely used in AI/ML

Python provides a readable language plus a mature ecosystem.

NumPy supports numerical arrays, Pandas supports tabular data, and Scikit-learn provides many classical ML algorithms and utilities.

Deep-learning and GenAI ecosystems also provide Python APIs.

2. Variables, Objects and Data Types

Variables and references

A Python variable is a name bound to an object.

Assignment does not necessarily copy the object. Multiple names can refer to the same object.

```
a = [1, 2]
b = a
b.append(3)
print(a) # [1, 2, 3]
```

Core built-in types

Common types include int, float, bool, str, list, tuple, set, dict and NoneType.

Use type() when you need to inspect an object's type.

```
x = 10
print(type(x))
```

Mutable vs immutable

Mutable objects can be changed after creation. Lists, dictionaries and sets are common mutable types.

Immutable objects cannot be changed in place. Strings, numbers and tuples are common immutable types.

This distinction matters when passing objects to functions and when discussing copying.

```
items = [1, 2]
items.append(3)

text = "python"
# text[0] = "P" # TypeError
```

is versus ==

== compares values for equality.

is checks whether two references point to the same object.

Do not use is as a replacement for normal value comparison.

```
a = [1, 2]
b = [1, 2]
print(a == b) # True
print(a is b) # False
```

3. Strings

String basics

Strings are immutable sequences of characters.

Important operations include indexing, slicing, split, join, strip, replace, lower and upper.

```
text = " Python ML "
print(text.strip())
print(text.lower())
print(text.split())
```

String formatting

Formatted string literals (f-strings) provide a readable way to insert values into strings.

```
name = "Omkar"
score = 8
print(f"{name} scored {score}/10")
```

Interview point

Because strings are immutable, operations that appear to modify a string actually produce a new string.

4. Lists, Tuples, Sets and Dictionaries

List

Lists are ordered and mutable.

Use a list when order matters and the collection may change.

```
skills = ["Python", "SQL"]
skills.append("ML")
```

Tuple

Tuples are ordered and immutable.

They are useful for fixed groups of values and can be used as dictionary keys when their elements are hashable.

```
point = (10, 20)
```

Set

Sets store unique elements and are useful for membership testing and removing duplicates.

```
skills = {"Python", "Python", "SQL"}
print(skills)
```

Dictionary

Dictionaries map keys to values.

Keys must be hashable. Dictionaries are useful when you need lookup by a meaningful key.

```
candidate = {"name": "Omkar", "role": "ML Engineer"}
print(candidate["name"])
```

Common interview comparison

List: ordered, mutable, duplicates allowed.

Tuple: ordered, immutable, duplicates allowed.

Set: unique elements, membership-oriented.

Dictionary: key-value mapping.

5. Operators and Expressions

Important operator groups

Arithmetic: +, -, *, /, //, %, **.

Comparison: ==, !=, <, <=, >, >=.

Logical: and, or, not.

Membership: in, not in.

Identity: is, is not.

```
print(10 // 3) # 3
print(10 % 3) # 1
print(2 ** 3) # 8
```

Short-circuit evaluation

and and or can stop evaluating once the result is known.

This can be useful for conditional checks but should remain readable.

```
name = None
if name and name.strip():
print(name)
```

6. Conditions and Loops

Conditions

if, elif and else control conditional execution.

Conditions can combine comparisons and logical operators.

```
score = 75
if score >= 70:
```

```
print("Good")
else:
print("Needs improvement")
```

for loops

A for loop iterates over an iterable.

```
for skill in ["Python", "SQL", "ML"]:
print(skill)
```

while loops

A while loop continues while its condition is true. Make sure the loop can eventually terminate.

```
count = 0
while count < 3:
print(count)
count += 1
```

break, continue and pass

break exits the loop.

continue skips to the next iteration.

pass does nothing and is commonly used as a placeholder.

7. Functions

Function basics

Functions package reusable behavior.

Parameters appear in the definition; arguments are values supplied during a call.

```
def add(a, b):
return a + b

print(add(2, 3))
```

Default parameters

A parameter can have a default value.

Avoid mutable objects such as lists as default values when you intend a fresh object for every call.

```
def greet(name="Candidate"):
return f"Hello {name}"
```

*args and **kwargs

*args collects extra positional arguments into a tuple.

**kwargs collects extra keyword arguments into a dictionary.

```
def report(*args, **kwargs):
print(args)
print(kwargs)

report("Python", score=8)
```

Return versus print

print displays a value.

return sends a value back to the caller and allows the caller to reuse it.

```
def square(x):
return x * x

result = square(5)
```

8. Scope and LEGB

LEGB

Python resolves a name through Local, Enclosing, Global and Built-in scopes.

Local is the current function, Enclosing refers to outer function scopes, Global is module scope, and Built-in contains names such as len and print.

global and nonlocal

global refers to a module-level variable.

nonlocal refers to a variable in an enclosing function scope.

```
x = 10

def show():
print(x)

show()
```

9. Comprehensions

List comprehension

A list comprehension creates a list from an iterable in a compact form.

Use it when it remains readable; a normal loop is often better for complex logic.

```
squares = [x * x for x in range(5)]
```

Dictionary and set comprehensions

Python also supports dictionary and set comprehensions.

```
squares = {x: x*x for x in range(3)}
unique = {x % 2 for x in range(5)}
```

10. Object-Oriented Programming

Class and object

A class describes data and behavior. An object is an instance of a class.

```
class Candidate:
def __init__(self, name):
self.name = name

candidate = Candidate("Omkar")
```

Encapsulation

Encapsulation combines related state and behavior and provides a controlled interface.

Python uses conventions such as a leading underscore to indicate internal attributes; it does not enforce private access in the same way as some languages.

Inheritance

Inheritance allows a class to reuse or extend behavior from another class.

```
class Engineer:
def work(self):
return "Building"

class MLEngineer(Engineer):
pass
```

Polymorphism

Different objects can provide the same interface while implementing behavior differently.

```
class Cat:
def speak(self):
return "meow"

class Dog:
def speak(self):
return "bark"

for animal in [Cat(), Dog()]:
print(animal.speak())
```

Abstraction

Abstraction focuses on what an object should expose while hiding implementation details that callers do not need.

11. Exception Handling

try / except / else / finally

try contains code that may fail.

except handles a matching exception.

else runs when no exception occurs.

finally runs whether an exception occurred or not.

```
try:
value = int("10")
except ValueError:
print("Invalid")
else:
print(value)
finally:
print("Finished")
```

Good exception handling

Catch specific exceptions when possible.

Do not silently swallow errors unless there is a clear reason.

Use exceptions for exceptional conditions rather than ordinary control flow.

12. Modules, Packages and Imports

Module

A module is a Python file that can be imported.

```
import math
print(math.sqrt(16))
```

Package

A package organizes modules into a reusable project structure.

Import style

Prefer clear imports that make dependencies easy to understand.

Avoid wildcard imports such as from module import * in production code because they make names harder to track.

13. Virtual Environments and Dependencies

Why virtual environments?

A virtual environment isolates project dependencies.

This prevents one project's package versions from unnecessarily affecting another project.

Requirements file

A requirements.txt file records dependencies so an environment can be reproduced more easily.

```
pip install -r requirements.txt
```

Your project connection

Your FastAPI backend uses a project-specific venv and requirements.txt. This is directly relevant to explaining your software project in an interview.

14. File Handling

Reading and writing

Use open() with a context manager so the file is closed automatically.

```
with open("notes.txt", "r", encoding="utf-8") as file:
text = file.read()
```

Common modes

r reads, w writes and replaces existing content, a appends, and x creates a new file when it does not exist.

15. Iterables, Iterators and Generators

Iterable

An iterable is an object you can iterate over, such as a list or string.

Iterator

An iterator produces values one at a time through the iterator protocol, using __iter__ and __next__.

```
numbers = iter([1, 2, 3])
print(next(numbers))
print(next(numbers))
```

Generator

A generator produces values lazily, usually with yield.

Generators can avoid keeping a complete result collection in memory.

```
def numbers(n):
for i in range(n):
yield i
```

```
for n in numbers(3):
print(n)
```

16. Lambda, map, filter and reduce

lambda

A lambda creates a small anonymous function.

```
double = lambda x: x * 2
print(double(5))
```

map and filter

map applies a function to each item.

filter keeps items for which the condition is true.

```
numbers = [1, 2, 3, 4]
print(list(map(lambda x: x*2, numbers)))
print(list(filter(lambda x: x % 2 == 0, numbers)))
```

reduce

reduce repeatedly combines values into one result and is provided by functools.

```
from functools import reduce
print(reduce(lambda a, b: a + b, [1, 2, 3]))
```

17. Copying Objects

Assignment is not copying

b = a creates another reference to the same object.

```
a = [1, 2]
b = a
b.append(3)
print(a)
```

Shallow copy

A shallow copy creates a new outer object while nested objects may still be shared.

```
a = [[1, 2], [3, 4]]
b = a.copy()
b.append([5, 6])
```

Deep copy

A deep copy recursively copies nested objects. The copy module provides deepcopy().

```
import copy
b = copy.deepcopy(a)
```

18. Decorators and Context Managers

Decorator

A decorator wraps a function or class to add or change behavior without modifying the original implementation directly.

```
def log_call(func):
def wrapper(*args, **kwargs):
print("Calling function")
```

```
return func(*args, **kwargs)
return wrapper

@log_call
def greet():
return "Hello"
```

Context manager

Context managers manage setup and cleanup around a block of code. The with statement is the common interface.

```
with open("notes.txt", "r", encoding="utf-8") as file:
text = file.read()
```

19. Python Memory and Performance

References

Names reference objects. Understanding references helps explain mutation, copying and function arguments.

Time complexity

Know the rough complexity of common operations and avoid unnecessary repeated work.

For interviews, be ready to explain why a dictionary or set can often provide efficient average-case membership lookup.

Memory considerations

Generators can be useful for large streams because they produce values lazily.

Do not optimize blindly; measure bottlenecks when performance matters.

20. Python in Machine Learning

Typical workflow

A Python ML workflow often includes data loading, cleaning, preprocessing, feature engineering, training, evaluation and inference.

NumPy

NumPy provides arrays and vectorized numerical operations and is a foundation for many Python data libraries.

Pandas

Pandas provides DataFrame and Series structures used for data inspection, cleaning, transformation and analysis.

Scikit-learn

Scikit-learn provides preprocessing, classical ML algorithms, model selection and evaluation utilities.

Interview connection

Be ready to explain exactly where you used Python, Pandas, NumPy and Scikit-learn in your own ML project rather than giving only textbook definitions.

21. Python Backend and FastAPI Connection

API concept

An API allows software components to communicate through defined requests and responses.

FastAPI uses Python type hints and Pydantic models for request validation and structured API development.

Your project connection

In the interview system, FastAPI receives resume uploads, validates request data and connects API routes to service-layer logic.

Separating API routes, schemas and services makes the code easier to test and maintain.

22. Common Python Interview Questions

Fundamentals

• What is Python?

• Why is Python dynamically typed?

• Is Python interpreted or compiled?

• What makes Python useful for AI/ML?

• What is the difference between a variable and an object?

Data Structures

• List vs tuple?

• Set vs dictionary?

• Why must dictionary keys be hashable?

• What is the difference between mutable and immutable objects?

• How would you remove duplicates from a list?

Functions and Scope

• What is the difference between parameters and arguments?

• What are *args and **kwargs?

• What is the LEGB rule?

• What is a closure?

• Why should mutable default arguments be avoided?

OOP

• What is a class and what is an object?

• Explain encapsulation, inheritance, polymorphism and abstraction.

• What is method overriding?

• When would you use composition instead of inheritance?

Exceptions

• How does try/except work?

• What is the purpose of finally?

• Why catch specific exceptions?

• What happens when an exception is not handled?

Advanced Python

• What is an iterator?

• What is a generator?

• What is a decorator?

• What is a context manager?

• Shallow copy vs deep copy?

AI/ML Python

- How are NumPy and Pandas different?
- What is a DataFrame?
- Why is vectorization useful?
- Where does Scikit-learn fit into an ML workflow?
- Explain one Python concept you used in your own ML project.

23. Coding Practice

Reverse a string

```
def reverse_text(text):
return text[::-1]
```

Count frequencies

```
def count_items(items):
counts = {}
for item in items:
counts[item] = counts.get(item, 0) + 1
return counts
```

Find duplicates

```
def duplicates(items):
seen = set()
result = set()
for item in items:
if item in seen:
result.add(item)
else:
seen.add(item)
return result
```

Check palindrome

```
def is_palindrome(text):
cleaned = text.lower()
return cleaned == cleaned[::-1]
```

Find maximum without max()

```
def find_max(numbers):
if not numbers:
raise ValueError("Empty list")
current = numbers[0]
for number in numbers[1:]:
if number > current:
current = number
return current
```

Generator practice

```
def even_numbers(n):
for i in range(n + 1):
if i % 2 == 0:
yield i
```

24. Project-Based Interview Practice

• Explain the architecture of your AI Interview Preparation System.

• Why did you choose FastAPI for the backend?

• Why did you separate API routes, schemas and services?

• How does resume upload move from the frontend to the backend?

• Why do you validate resume input before sending it to Gemini?

• Why use a structured candidate profile instead of keeping the Gemini response as raw text?

• Why do we need RAG if an LLM can already answer questions?

• What are embeddings?

• What is a vector database?

• What is chunking and why is overlap useful?

• What is top-k retrieval?

• How would you prevent irrelevant chunks from entering the LLM prompt?

• How would you evaluate whether your RAG system retrieves useful information?

• What happens if the Gemini API quota is exhausted?

• How would you make the system more reliable in production?

25. How to Answer in an Interview

Definition question

Define the concept → explain how it works → give a small example → mention a practical use.

Comparison question

Define both → state the important difference → explain when each is appropriate → give an example.

Coding question

Clarify the requirement → explain the approach → code → test an example → discuss complexity if relevant.

Project question

Problem → architecture/approach → technologies → your contribution → challenge → result → possible improvement.

When you do not know

Do not invent an answer. State what you know, identify the gap, and explain how you would verify or learn the missing part.

26. Revision Checklist

■ Explain Python dynamic typing without saying only 'Python is interpreted'.

■ Explain mutable vs immutable objects with an example.

■ Explain list vs tuple vs set vs dictionary.

■ Explain == vs is.

■ Explain *args and **kwargs.

■ Explain LEGB.

■ Explain the four OOP concepts.

■ Explain exception handling.

■ Explain virtual environments.

■ Explain iterables, iterators and generators.

■ Explain decorators and context managers.

■ Explain shallow vs deep copy.

■ Explain NumPy, Pandas and Scikit-learn.

■ Explain how Python is used in your own AI/ML project.

■ Answer at least 10 coding questions without copying the solution.

27. RAG Knowledge-Base Note

This document is intentionally organized into focused sections. When we build the RAG pipeline, we should inspect the extracted text and choose chunking parameters based on actual content rather than page count alone. The goal is for a retrieved chunk to contain a complete, useful concept that can support an interview question.

> A larger document is not automatically a better RAG document. Retrieval quality depends on content quality, chunk boundaries, embeddings, metadata, similarity search and evaluation. We will measure these stages as we build the system.